

编译原理-语法分析 04

预测分析法

授课教师：柯嵩宇

福州大学计算机与大数据学院

2025 年 4 月 1 日



福州大学
FUZHOU UNIVERSITY 1958



- ① 课前复习
- ② LL(1) 文法
- ③ 预测分析法

③ 预测分析法

编译过程的重要步骤

编译过程的主要步骤:

① 词法分析

- 输入：字符流；
- 输出：词元流、符号表；

② 语法分析

- 输入: 词元流、符号表;
- 输出: 语法树、符号表;

③ 语义分析

- 输入：语法树、符号表；
- 输出：修饰后的语法树；

④ 中间代码生成

- 输入: 修饰后的语法树;
- 输出: 中间表示 IR;

⑤ 机器无关优化

- 输入: IR;
- 输出: IR;

⑥ 机器相关优化

- 输入: IR;
- 输出: IR;

⑦ 代码生成

- 输入：IR;
- 输出：汇编、字节码;

词法分析器

正则文法

正则文法 $\mathcal{G} = (\mathbb{N}, \Sigma, \mathbb{P}, S)$, 其中文法

- 产生式规则集 $\mathbb{P}$, 每个规则都为如下形式

$$N \rightarrow \Sigma^* N,$$

即正则文法的产生式左侧仅允许存在一个非终结符, 右侧终结符必须在非终结符前面。

- 不是一般性地, 我们可以要求只有一个非终结符在右侧的产生式中。

上下文无关文法

上下文无关文法 $\mathcal{G} = (\mathbb{N}, \Sigma, \mathbb{P}, S)$, 其中文法

- 产生式规则集 $\mathbb{P}$, 每个规则都为如下形式

$$\mathbb{N} \rightarrow (\Sigma \cup \mathbb{N})^*,$$

即上下文无关文法的产生式左侧仅允许存在一个非终结符。

语法分析器的类型

自顶向下 (Top-Down)

- 从**开始符号**出发, 寻找“匹配”的**推导**。
- 推导: 根据文法的产生式规则, 把串中出现的产生式的左部符号替换成右部。
- 从**根**开始, 构造语法树。
- 具体实现: 递归下降分析法、预测分析法。

自底向上 (Bottom-Up)

- 从**输入串**开始, 逐步进行**归约**, 直到文法的**开始符号**。
- 归约: 根据文法的产生式规则, 把串中出现的产生式的右部替换成左部符号。
- 从**叶节点**开始, 构造语法树。
- 具体实现: 算符优先分析法、LR 分析法。

Top-Down 分析器挑战一：左递归

在自顶向下分析器寻找匹配的推导时，通常采用的是最左推导的形式，即每次选择**最左边**的非终结符进行替换。

按顺序尝试非终结符的不同产生式，有可能出现最左侧符号不断出现的**无限循环**的情况。

文法

$expr \rightarrow expr + term | expr - term | term$

$term \rightarrow term * factor | term / factor | factor$

$factor \rightarrow digit | (expr)$

$digit \rightarrow 0 | 1 | \dots | 9$

目标句子：8+8+8

$expr \Rightarrow expr + term$

$\Rightarrow expr + term + term$

$\Rightarrow expr + term + term$

$\Rightarrow expr + term + term + term$

$\Rightarrow expr + term + term + term + term$

Top-Down 分析器挑战一：左递归

左递归的两种形式：

直接左递归

$$\begin{aligned} \text{expr} &\rightarrow \text{expr} + \text{term} \mid \text{expr} - \text{term} \mid \text{term} \\ \text{term} &\rightarrow \text{term} * \text{factor} \mid \text{term} / \text{factor} \mid \text{factor} \\ \text{factor} &\rightarrow \text{digit} \mid (\text{expr}) \\ \text{digit} &\rightarrow 0 \mid 1 \mid \dots \mid 9 \end{aligned}$$

间接左递归

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow TC \\ T &\rightarrow S\gamma \end{aligned}$$

Top-Down 分析器挑战二：回溯

- 分析过程中，当一个非终结符用某一个候选式匹配成功时，这种匹配可能是暂时的。出错时，需要进行“回溯”。
- 当同一个非终结符的多个候选式存在共同前缀时，将导致回溯。

Top-Down 分析器挑战二：回溯

消除回溯

- 目标：对文法的任何非终结符，当要它去匹配输入串时，能够根据它所面临的输入符号准确地指派它的一个候选去执行任务，并且此候选的工作结果应是确信无疑的。
- 方法：提取左公因子

Top-Down 分析器挑战二：回溯

如下文法该选择哪条规则？

$$\begin{aligned} stmt &\rightarrow if (expr) then stmt else stmt \\ &| if (expr) then stmt \end{aligned}$$

Top-Down 分析器挑战二：回溯

如下文法该选择哪条规则？

$$\begin{aligned} stmt &\rightarrow \underline{if (expr) then stmt else stmt} \\ &\quad | \quad \underline{if (expr) then stmt} \end{aligned}$$

手段：通过改写产生式来推迟决定，等读入了足够多的输入，获得足够信息后再做出正确的选择。

$$\begin{aligned} stmt &\rightarrow if (expr) then stmt else stmt \\ elsestmt &\rightarrow \varepsilon \quad | \quad else stmt \end{aligned}$$

提取左公因式

数量优先

例

改写文法 $S \rightarrow abcd|abce|abf$ 以便无回溯的自顶向下分析。

解

S 所有子式的最长左公因子为 ab , 引入 A 改写生成式得 $S \rightarrow abA, A \rightarrow cd|ce|f$ 。

再提取 A 的左公因子并引入 B 改写生成式得 $A \rightarrow cB|f, B \rightarrow d|e$ 。

最终文法为

$$S \rightarrow abA$$

$$A \rightarrow cB|f$$

$$B \rightarrow d|e$$

文法处理的顺序

① 消除左递归;



文法处理的顺序

- ① 消除左递归;
- ② 提取左公因子以消除回溯;



文法处理的顺序

- ① 消除左递归;
- ② 提取左公因子以消除回溯;
- ③ 处理文法的二义性;



递归下降分析法

典型过程

Algorithm 1: 处理非终结符号的典型过程

```
1 void A () {  
2   选择一个 A 的产生式,  $A \rightarrow X_1, X_2, \dots, X_k$ ;  
3   for  $i \leftarrow 1$  to  $k$  do  
4     if  $X_i$  是一个非终结符号 then  
5       调用过程  $X_i()$ ;  
6     else  
7       if  $X_i$  等于当前的输入符号  $a$  then  
8         继续读取下一个输入符号;  
9       else  
10        处理错误;  
11      end  
12 end
```

该方法的问题

- ① 未说明如何选择产生式（不确定，乱选瞎选）。
- ② 可能需要回溯（降低效率）。
- ③ 可能出现无限循环。



扩展知识之 PEG

解析表达式文法 (Parsing Expression Grammar) 是一种描述语法规则的形式化方法, 专为解析而设计。相比 CFG, PEG 更关注如何通过规则匹配输入文本, 而非生成语言结构。

两种方法并非完全对立, 现代工具 (如 ANTLR 4) 允许混合使用 CFG 与 PEG 特性 (如优先规则)。

例

```
Expr ← AddExpr
AddExpr ← MulExpr (('+' / '-') MulExpr)*
MulExpr ← Primary (('*' / '/') Primary)*
Primary ← Number / '(' Expr ') '
Number ← [0-9]+
```

扩展知识之 PEG

PEG 与 CFG 的核心差异对比

特性	CFG (上下文无关文法)	PEG (解析表达式文法)
设计目标	描述语言生成规则 (生成式)	描述解析过程 (识别式)
规则定义	声明式 (规则独立, 顺序无关)	过程式 (规则顺序影响匹配优先级)
歧义性	允许歧义, 需额外消歧 (如运算符优先级)	天然无歧义 (通过优先级顺序隐式消歧)
回溯机制	通常不回溯 (需预测式解析器如 LL/LR)	支持贪婪回溯 (匹配失败时尝试其他分支)
词法独立	需要独立词法分析阶段 (如正则表达式分词)	词法和语法可统一描述 (无严格分离)
左递归支持	支持 (需 LR 解析器)	原生不支持 (需改写或借助工具扩展)

扩展知识

语法分析的最坏情况

请回答：语法分析过程的最坏时间复杂度是多少？



扩展知识

语法分析的最坏情况

请回答：语法分析过程的最坏时间复杂度是多少？

$$\mathcal{O}(n^3)$$



扩展知识

语法分析的最坏情况

请回答：语法分析过程的最坏时间复杂度是多少？

$\mathcal{O}(n^3)$ 如何证明？

① 改写文法；

扩展知识

语法分析的最坏情况

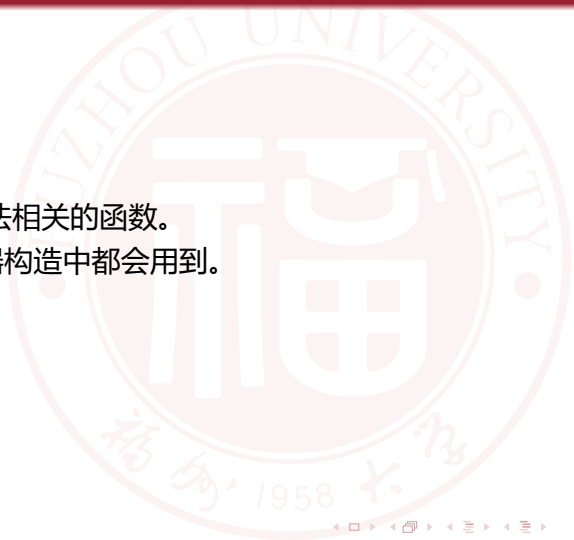
请回答：语法分析过程的最坏时间复杂度是多少？

$\mathcal{O}(n^3)$ 如何证明？

- ① 改写文法；
- ② 动态规划；

FIRST 与 FOLLOW

- FIRST 和 FOLLOW 是两个文法相关的函数。
- 在自顶向下与自底向上的分析器构造中都会用到。



FIRST

定义

$\text{FIRST}(\alpha)$ 是可以由 α 推导得到的串首符号的集合, 其中 α 是任意的文法符号串, 具体地

- $\forall \alpha \in (\mathbb{V}_T \cup \mathbb{V}_N)^+, \text{FIRST}(\alpha) = \{a | \alpha \Rightarrow^* a\beta, a \in \mathbb{V}_T, \beta \in (\mathbb{V}_T \cup \mathbb{V}_N)^*\}$
- 若 $\alpha \Rightarrow^* \varepsilon$ 合法, 则 $\varepsilon \in \text{FIRST}(\alpha)$

推论

如果非终结符 A 的所有候选式的首终结符集两两不相交, 当要求 A 匹配输入串时, A 就能根据当前输入符号 c , 选择唯一的正确的候选式, 即串首终结符集中包含 c 的那个候选式。

计算 FIRST 的算法

Algorithm 2: 计算语法符号 X 的串首集 $\text{FIRST}(X)$ 的算法

Input: 语法符号 X

Output: 串首集 $\text{FIRST}(X)$

```

1  if  $X \in \mathbb{V}^T$  then
2     $\text{FIRST}(X) \leftarrow \{X\};$ 
3  else
4     $\text{FIRST}(X) \leftarrow \emptyset;$ 
5    for all  $X \rightarrow Y_1 Y_2 \dots Y_k \in \mathbb{P}$  do
6      for  $i \leftarrow 1$  to  $k$  do
7        if  $a \in \text{FIRST}(Y_i) \wedge \forall j < i, \varepsilon \in \text{FIRST}(Y_j)$  then
8           $\text{FIRST}(X) \leftarrow \text{FIRST}(X) \cup \{a\};$ 
9        end
10       end
11       if  $\forall j \leq i, \varepsilon \in \text{FIRST}(Y_j)$  then
12         end
13       end
14  end
    
```

FOLLOW

- 对于非终结符号 A , $\text{FOLLOW}(A)$ 是所有在某些句型中紧跟在 A 右边的终结符号的集合。
- A 与 a 之间可能存在其他的非终结符, 但都会随着推导变成 ϵ 。
- 特别地, 如果 A 是某些句型最右的符号, 则 $\$ \in \text{FOLLOW}(A)$, 其中 $\$$ 表示输入的终止 (EOF)。

例

如果存在 $S \Rightarrow^* \alpha A a \beta$ 的推导, 则 $a \in \text{FOLLOW}(A)$ 。

计算 FOLLOW 的算法

Algorithm 3: 计算所有非终结符紧随集 FOLLOW 的算法

Input: 语法符号 A

Output: 紧随集 FOLLOW

```
1 FOLLOW( $S$ )  $\rightarrow$   $\{\$ \}$ ;  
2 repeat  
3   对于所有产生式判定如下条件:  
4   if  $A \rightarrow \alpha B \beta \in \mathbb{P}$  then  
5     FOLLOW( $B$ )  $\leftarrow$  FOLLOW( $B$ )  $\cup$  (FIRST( $B$ )  $\setminus$   $\{\epsilon\}$ );  
6   end  
7   if  $A \rightarrow \alpha B \beta \in \mathbb{P}$  or  $A \rightarrow \alpha B \beta \in \mathbb{P} \wedge \epsilon \in \text{FIRST}(\beta)$  then  
8     FOLLOW( $B$ )  $\leftarrow$  FOLLOW( $B$ )  $\cup$  FOLLOW( $A$ );  
9   end  
10 until 所有非终结符的紧随集不再发生变化;
```

计算文法符号 X 的 FOLLOW(X)

例

计算下列文法的 FOLLOW 集:

$$E \rightarrow TE'$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FOLLOW}(E) = \{ \$,) \}$$

$$E' \rightarrow +TE' | \varepsilon$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FOLLOW}(E') = \{ \$,) \}$$

$$T \rightarrow FT'$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FOLLOW}(T) = \{ +, \$,) \}$$

$$T' \rightarrow *FT' | \varepsilon$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FOLLOW}(T') = \{ +, \$,) \}$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(F) = \{ (, id \}$$

$$\text{FOLLOW}(F) = \{ *, +, \$,) \}$$

练习

计算下列文法的 FIRST 和 FOLLOW 集：

$$S \rightarrow AB|bC$$

$$A \rightarrow \varepsilon|b$$

$$B \rightarrow \varepsilon|aD$$

$$C \rightarrow AD|b$$

$$D \rightarrow aS|c$$

练习

计算下列文法的 FIRST 和 FOLLOW 集：

$$S \rightarrow AB|bC$$

$$A \rightarrow \varepsilon|b$$

$$B \rightarrow \varepsilon|aD$$

$$C \rightarrow AD|b$$

$$D \rightarrow aS|c$$

$$\text{FIRST}(S) = \{\varepsilon, a, b\}$$

$$\text{FIRST}(A) = \{\varepsilon, b\}$$

$$\text{FIRST}(B) = \{\varepsilon, a\}$$

$$\text{FIRST}(C) = \{a, b, c\}$$

$$\text{FIRST}(D) = \{a, b\}$$

$(A \rightarrow \alpha B \beta \in \mathbb{P}) \rightarrow (\text{FIRST}(\beta) \setminus \{\varepsilon\} \subseteq \text{FOLLOW}(B))$ 。

$(A \rightarrow aB \in \mathbb{P}) \vee (A \rightarrow \alpha B \beta \in \mathbb{P} \wedge \varepsilon \in \text{FIRST}(\beta)) \rightarrow \text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$ 。

练习

计算下列文法的 FIRST 和 FOLLOW 集：

$S \rightarrow AB|bC$

$\text{FIRST}(S) = \{\varepsilon, a, b\}$

$A \rightarrow \varepsilon|b$

$\text{FIRST}(A) = \{\varepsilon, b\}$

$B \rightarrow \varepsilon|aD$

$\text{FIRST}(B) = \{\varepsilon, a\}$

$C \rightarrow AD|b$

$\text{FIRST}(C) = \{a, b, c\}$

$D \rightarrow aS|c$

$\text{FIRST}(D) = \{a, b\}$

$(A \rightarrow \alpha B \beta \in \mathbb{P}) \rightarrow (\text{FIRST}(\beta) \setminus \{\varepsilon\} \subseteq \text{FOLLOW}(B))$ 。

$(A \rightarrow aB \in \mathbb{P}) \vee (A \rightarrow \alpha B \beta \in \mathbb{P} \wedge \varepsilon \in \text{FIRST}(\beta)) \rightarrow \text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$ 。

练习

计算下列文法的 FIRST 和 FOLLOW 集：

$S \rightarrow AB|bC$

$\text{FIRST}(S) = \{\varepsilon, a, b\}$

$\text{FOLLOW}(S) = \{\$ \}$

$A \rightarrow \varepsilon|b$

$\text{FIRST}(A) = \{\varepsilon, b\}$

$\text{FOLLOW}(A) = \{\$, a, c\}$

$B \rightarrow \varepsilon|aD$

$\text{FIRST}(B) = \{\varepsilon, a\}$

$\text{FOLLOW}(B) = \{\$ \}$

$C \rightarrow AD|b$

$\text{FIRST}(C) = \{a, b, c\}$

$\text{FOLLOW}(C) = \{\$ \}$

$D \rightarrow aS|c$

$\text{FIRST}(D) = \{a, b\}$

$\text{FOLLOW}(D) = \{\$ \}$

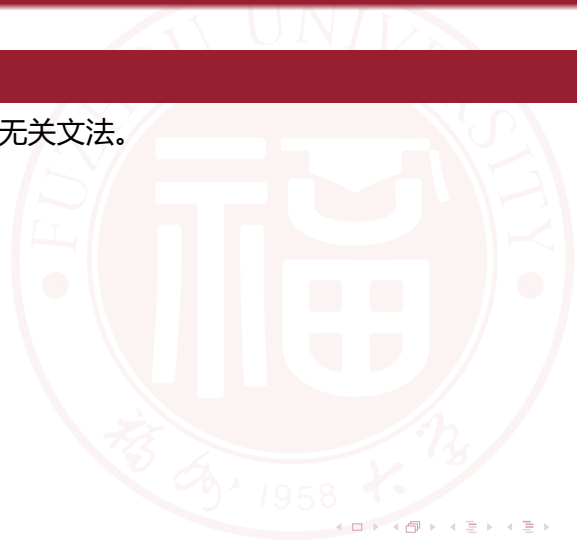


- ① 课前复习
- ② LL(1) 文法
- ③ 预测分析法

LL(1) 文法

定义

- LL(1) 文法是一类特别的上下文无关文法。



LL(1) 文法

定义

- LL(1) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；

LL(1) 文法

定义

- LL(1) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
 - 第二个 L 表示 “Leftmost”，即最左推导；

LL(1) 文法

定义

- LL(1) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
 - 第二个 L 表示 “Leftmost”，即最左推导；
 - 1 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。

LL(1) 文法

定义

- LL(1) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
 - 第二个 L 表示 “Leftmost”，即最左推导；
 - 1 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。
- 左递归文法和二义文法都不是 LL(1) 文法。

LL(1) 文法

定义

- LL(1) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
 - 第二个 L 表示 “Leftmost”，即最左推导；
 - 1 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。
- 左递归文法和二义文法都不是 LL(1) 文法。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。

LL(1) 文法

定义

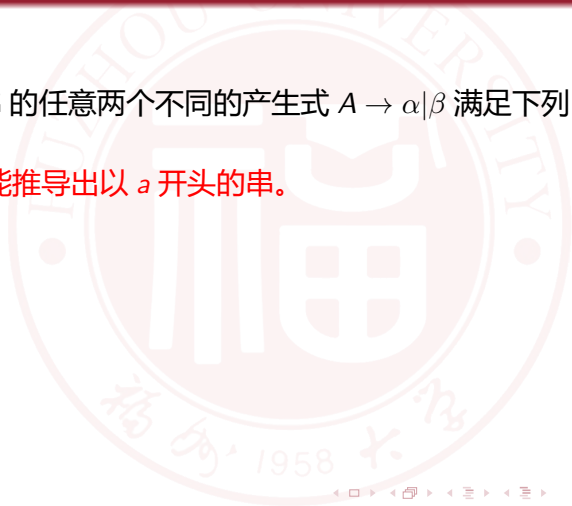
- LL(1) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
 - 第二个 L 表示 “Leftmost”，即最左推导；
 - 1 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。
- 左递归文法和二义文法都不是 LL(1) 文法。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。
- LL(1) 文法可以将 ϵ 边作为最后的默认选择以解决 ϵ 的二义性（例如，if 语句的 else 匹配问题）。

LL(1) 文法

G 是 LL(1) 文法的充分必要条件

一个文法 G 是 LL(1) 文法当且仅当 G 的任意两个不同的产生式 $A \rightarrow \alpha | \beta$ 满足下列条件

- ① 不存在终结符号 a 使 α 和 β 都能推导出以 a 开头的串。



LL(1) 文法

G 是 LL(1) 文法的充分必要条件

一个文法 G 是 LL(1) 文法当且仅当 G 的任意两个不同的产生式 $A \rightarrow \alpha | \beta$ 满足下列条件

- ① 不存在终结符号 a 使 α 和 β 都能推导出以 a 开头的串。
- ② α 、 β 中最多一个可以推导出空串 ε 。

LL(1) 文法

G 是 LL(1) 文法的充分必要条件

一个文法 G 是 LL(1) 文法当且仅当 G 的任意两个不同的产生式 $A \rightarrow \alpha | \beta$ 满足下列条件

- ① 不存在终结符号 a 使 α 和 β 都能推导出以 a 开头的串。
- ② α 、 β 中最多一个可以推导出空串 ε 。

LL(1) 文法

G 是 LL(1) 文法的充分必要条件

一个文法 G 是 LL(1) 文法当且仅当 G 的任意两个不同的产生式 $A \rightarrow \alpha | \beta$ 满足下列条件

- ① 不存在终结符号 a 使 α 和 β 都能推导出以 a 开头的串。
- ② α 、 β 中最多一个可以推导出空串 ε 。
(上面两个条件等价于 $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$)
- ③ 若 $\beta \Rightarrow^* \varepsilon$, 则 α 不能推导出任何以 $\text{FOLLOW}(A)$ 中某个终结符开头的串。反之亦然。

LL(1) 文法

G 是 LL(1) 文法的充分必要条件

一个文法 G 是 LL(1) 文法当且仅当 G 的任意两个不同的产生式 $A \rightarrow \alpha | \beta$ 满足下列条件

- ① 不存在终结符号 a 使 α 和 β 都能推导出以 a 开头的串。
- ② α 、 β 中最多一个可以推导出空串 ε 。
(上面两个条件等价于 $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$)
- ③ 若 $\beta \Rightarrow^* \varepsilon$, 则 α 不能推导出任何以 $\text{FOLLOW}(A)$ 中某个终结符开头的串。反之亦然。

LL(1) 文法

G 是 LL(1) 文法的充分必要条件

一个文法 G 是 LL(1) 文法当且仅当 G 的任意两个不同的产生式 $A \rightarrow \alpha | \beta$ 满足下列条件

- ① 不存在终结符号 a 使 α 和 β 都能推导出以 a 开头的串。
- ② α 、 β 中最多一个可以推导出空串 ε 。
(上面两个条件等价于 $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$)
- ③ 若 $\beta \Rightarrow^* \varepsilon$, 则 α 不能推导出任何以 $\text{FOLLOW}(A)$ 中某个终结符开头的串。反之亦然。
($\varepsilon \in \text{FIRST}(\beta) \rightarrow \text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$)

- ① 课前复习
- ② LL(1) 文法
- ③ 预测分析法

表驱动预测分析法

定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。

表驱动的预测分析法

定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。
- 小提问：对于每一个 NFA 都可以构造一个 DFA 与其等价，那所有的 PDA 都可以构造一个确定性的下推自动机与其等价吗？

表驱动的预测分析法

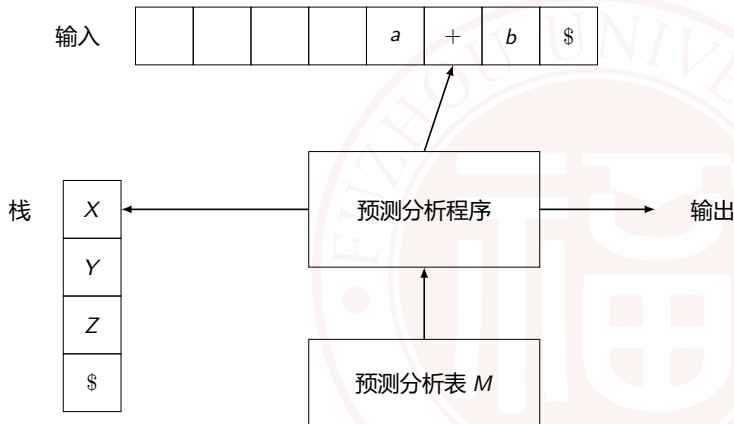
定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。
- 特征：不需要为每个非终结符编写递归下降过程，而是根据预测分析表构造一个不需要回溯的非递归下降语法分析器。

表驱动的预测分析法

定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。
- 特征：不需要为每个非终结符编写递归下降过程，而是根据预测分析表构造一个不需要回溯的非递归下降语法分析器。
- 要求：文法是 LL(1) 的。只有对于 LL(1) 文法，可以对输入串进行有效的无回溯的自上而下分析。



一个分析表驱动的分析器模型

预测分析表

定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
 - 行代表每一个非终结符；

预测分析表

定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
 - 行代表每一个非终结符；
 - 列代表每一个终结符（包括结束符号\$）；

预测分析表

定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
 - 行代表每一个非终结符；
 - 列代表每一个终结符（包括结束符号\$）；
 - 每个单元格里填写对应的产生式，或者错误标记；

预测分析表

定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
 - 行代表每一个非终结符；
 - 列代表每一个终结符（包括结束符号\$）；
 - 每个单元格里填写对应的产生式，或者错误标记；
 - 单元格 (i, j) 的内容表示若在匹配第 i 行的非终结符时遇到第 j 列的终结符时，该确定性地选用的生成式。

预测分析表

例

计算下列文法的预测分析表：

$$E \rightarrow TE'$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FOLLOW}(E) = \{ \$,) \}$$

$$E' \rightarrow +TE' | \varepsilon$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FOLLOW}(E') = \{ \$,) \}$$

$$T \rightarrow FT'$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FOLLOW}(T) = \{ +, \$,) \}$$

$$T' \rightarrow *FT' | \varepsilon$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FOLLOW}(T') = \{ +, \$,) \}$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(F) = \{ (, id \}$$

$$\text{FOLLOW}(F) = \{ *, +, \$,) \}$$

预测分析表

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

预测分析表

Algorithm 4: 预测分析表构造算法

Input: $\mathbb{P}$, $\text{FIRST}(\cdot)$, $\text{FOLLOW}(\cdot)$

```
1 for  $A \rightarrow \alpha \in \mathbb{P}$  do
2   if  $a \in \text{FIRST}(\alpha)$  then
3      $M[A, a] \leftarrow (A \rightarrow \alpha)$ ;
4   end
5   if  $\varepsilon \in \text{FIRST}(\alpha)$  then
6     for  $b \in \text{FOLLOW}(A)$  do
7        $M[A, b] \leftarrow (A \rightarrow \alpha)$ ;
8     end
9   end
10 end
11 剩余空格打上错误标记;
12 如果一个格子里面有多于 1 条规则, 不是 LL(1) 文法, 返回错误;
```

表驱动的预测分析法

Algorithm 5: 表驱动的预测分析法

Input: 输入串 w , 文法 $\mathcal{G}$ 的预测分析表 M

Output: 如果 $w \in \mathcal{L}(\mathcal{G})$, 输出 w 的一个最左推导, 否则输出错误提示

```

1  $X \equiv$  栈顶符号;  $ip$  指向当前处理的输入符号;
2 while  $X \neq \$/*$  栈非空  $*/$  do
3   if  $X = *ip$  then
4      $\text{pop}(X); ip++;$ 
5   else if  $X \in V_T$  or  $M[X, *ip]$  是一条报错条目 then
6     错误处理;
7   else if  $M[X, *ip] = X \rightarrow Y_1, Y_2, \dots, Y_k$  then
8     输出  $X \rightarrow Y_1, Y_2, \dots, Y_k;$ 
9      $\text{pop}(X);$ 
10     $\text{push}(Y_k); \text{push}(Y_{k-1}); \dots; \text{push}(Y_1);$ 
11  end
12 end
```

例

写出文法的预测分析表：

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \varepsilon$$

$$E \rightarrow b$$

例

写出文法的预测分析表：

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \varepsilon$$

$$E \rightarrow b$$

推论

如果冲突的是生成 ε 的产生式，那么可以将 ε 产生式丢弃。

预测分析表

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

例

基于上面的预测分析表，尝试解析 $(id * id) + id + id * id$ 。

预测分析法

错误恢复

如何检测错误?



预测分析法

错误恢复

如何检测错误?

- ① 栈顶的终结符号与输入不匹配;



预测分析法

错误恢复

如何检测错误?

- ① 栈顶的终结符号与输入不匹配;
- ② 栈顶的非终结符号 A , 但 $M[A, a] = error$ 。



预测分析法

错误恢复

如何检测错误?

- ① 栈顶的终结符号与输入不匹配;
- ② 栈顶的非终结符号 A , 但 $M[A, a] = error$ 。



预测分析法

错误恢复

如何检测错误?

- ① 栈顶的终结符号与输入不匹配;
- ② 栈顶的非终结符号 A , 但 $M[A, a] = error$ 。

如何处理错误?

- ① 恐慌模式 (Panic mode): 忽略输入流中的一系列符号, 直到出现符合设计者预期的词元;
- ② 短语恢复: 改变、插入或删除输入中的符号, 并给出适当的错误信息。

恐慌模式的错误处理

定义

同步集合 (synchronization set): 错误处理函数/过程会不断丢弃输入的符号直到遇到一个同步集合内的符号。

恐慌模式的错误处理

定义

同步集合 (synchronization set): 错误处理函数/过程会不断丢弃输入的符号直到遇到一个同步集合内的符号。

如何构造同步集合?

- ① 将 FOLLOW(A) 里的元素都加入同步集合;

恐慌模式的错误处理

例

针对文法：

`assign_stmt := expression ;`

输入：a+b+;

因为 $;\in \text{FOLLOW}(\text{expression})$ ，在其同步集合中，故尝试丢弃+后继续分析。

恐慌模式的错误处理

例

针对文法：

`assign_stmt := expression ;`

输入：`a+b+`;

因为 $;\in \text{FOLLOW}(\text{expression})$ ，在其同步集合中，故尝试丢弃+后继续分析。

够了吗？

例 (反例)

`a = 100 if (b != a) ...`

恐慌模式的错误处理

定义

同步集合 (synchronization set): 错误处理函数/过程会不断丢弃输入的符号直到遇到一个同步集合内的符号。

如何构造同步集合?

- ① 将 FOLLOW(A) 里的元素都加入同步集合;
- ② 将更高层次的非终结符号的 FOLLOW 集合也加入同步集合中;
- ③ 将 FIRST(A) 的符号也加入同步集合; **WHY?**
- ④ 如果非终结符可以生成 ϵ , 则使用它作为默认值;
- ⑤ 直接把不匹配的终结符号弹出;